

Design and Development of a Digital Tutor as a Mediating Interface for Innovation Wing Resource Access

Submitted by

Bao Xinyang

3035952989

B. Eng. (University of Hong Kong)

A dissertation submitted in partial fulfilment of the requirements for the degree of

Master of Science in Engineering

(Electrical and Electronic Engineering)

At the

Department of Electrical and Computer Engineering

The University of Hong Kong

In

May 2026

ABSTRACT

University makerspaces support project-based engineering education by providing students with various resources. However, students may struggle to translate project goals into suitable resources, procedures, and actionable workflows. This work focuses on improving how students discover and utilise makerspace resources during project development.

The developed system uses a modular LLM-based workflow consisting of request understanding, task planning, tool execution and response generation. Simple requests can be handled directly follow a lightweight execution path, while more complex requests are represented as task graphs with different dependency structures.

The system was evaluated using 100 requests in the situation of HKU Innovation Wing scenario. Module-level experiments examined request understanding, planning, and execution performance. End-to-end experiments compared the proposed approach with two simpler tool-using baselines. The results indicate that the system can effectively support multi-step resource-related requests and maintain traceable intermediate information during execution.

This study shows that modular LLM workflows can provide a practical approach for Digital Tutors in makerspace environments, where students require not only answers but also guidance on how to locate and utilise appropriate institutional resources.

Abstract Word Count: 174

DECLARATION

I declare that this dissertation represents my own work, except where due acknowledgement is made, and that it has not been previously included in a thesis, dissertation or report submitted to this University or to any other institution for a degree, diploma or other qualifications

Signed____Bao Xinyang_____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Prof. Yulin Shao, for his continuous guidance, encouragement, and valuable advice throughout this dissertation. His insightful suggestions and constructive feedback pointed me in the right direction when I encountered bottlenecks during this project.

I would also like to thank the PhD students in Prof. Shao's research group for their generous support and helpful discussions during this project. Their valuable suggestions, technical assistance, and willingness to share their research experience have provided me with great support throughout the development and evaluation of this work.

Finally, I would like to thank everyone who has supported me during my master's life. Your encouragement and support have made this journey more enjoyable and meaningful.

Table of Content

ABSTRACT	iii
DECLARATION	iii
ACKNOWLEDGEMENT	iv
Table of Content	v
List of Tables	vii
List of Figures	viii
Chapter 1 Introduction	1
1.1 Background	1
1.2 Motivation	4
1.3 Outline	7
Chapter 2 Literature Review	8
2.1 LLM-Based Digital Tutors in Education	8
2.2 Task-Oriented LLM Agents and Task Decomposition	10
2.3 Multi-Agent Orchestration and State-Driven Workflow	13
2.4 Research Gap	16
Chapter 3 Methodology	17
3.1 System Requirements and Design Objectives	17
3.1.1 Application Requirements	17
3.1.2 Functional Requirements	18
3.1.3 Design Objectives	19
3.2 Overall System Architecture	20
3.3 Request Understanding and Control Layer	21
3.4 Planning Layer	22
3.4.1 Task Decomposition and Dependency Planning	22
3.4.2 Dependency Graph Construction	23
3.4.3 Plan Validation	24
3.4.4 Plan-structure Classification	25
3.5 Execution Layer	26
3.6 Response Generation	28
3.6.1 Evidence Preparation	28

3.6.2 Grounded Response Synthesis	28
Chapter 4 Experiments and Evaluation	30
4.1 Research Questions.....	30
4.2 Evaluation Dataset.....	33
4.3 Experimental Design	34
4.3.1 Compared Systems and Baselines.....	35
4.3.2 Implementation Settings	37
4.3.3 Evaluation Metrics	39
4.4 Module-Level Evaluation Results	44
4.4.1 Understanding and Control Layer.....	44
4.4.2 Planning Layer	46
4.4.3 Execution Layer.....	48
4.5 End-to-End Performance and Baseline Comparison	48
4.5.1 Overall End-to-End Comparison	49
4.5.2 Performance by Single/Complex Request.....	50
4.5.3 Efficiency and Cost	52
4.6 Error Analysis	53
4.6.1 Intent and Routing Errors	53
4.6.2 Planning and Dependency Errors	54
4.6.3 Tool Execution and Input Adaptation Errors	55
4.6.4 Response Generation Errors	56
4.7 Discussion of Results.....	57
Chapter 5 Conclusion	61
5.1 Summary of the Study	61
5.2 Main Findings	61
5.3 Limitations	62
5.4 Future Work.....	63
Appendix A. Project Repository	65
References	66

List of Tables

Table 1 Task-to-Tool Mapping and Main Outputs	27
Table 2 Summary of Research Questions and Evaluation Measures	33
Table 3 Settings for final comparison	38
Table 4 Summary of error patterns.	57

List of Figures

Figure 1 Proposed System Architecture.....21

Figure 2 Performance of the Understanding and Control Layers45

Figure 3 Performance of the planning layer.....47

Figure 4 Performance of the execution layer.48

Chapter 1 Introduction

1.1 Background

Engineering education is increasingly organised around a central expectation: students should be able not only to understand engineering principles, but also to apply and integrate them in authentic, open-ended problem contexts. A systematic review of problem-, project-, and challenge-based learning describes active learning as an emerging “new norm” in engineering education, with university educators increasingly expected to replace predominantly teacher-centred knowledge transmission with learning activities in which students investigate, design, construct, and evaluate possible responses to engineering problems [1]. This shift is also evident in undergraduate curricula, where experiential learning activities—particularly engineering design projects—have been increasingly incorporated to connect disciplinary knowledge with design, implementation, testing, and reflection [2]. The growing significance of this pedagogical model is further reflected in the research landscape. A bibliometric analysis of 2,273 project-based learning publications from 2014 to 2024 reported an approximately eightfold increase in publication volume, with higher education, engineering education, and STEM among the most prominent application contexts [3]. These developments establish the ability to apply knowledge through experimentation, collaboration and problem-solving as a core outcome of contemporary engineering education.

Universities are supporting this transformation not only by redesigning their curricula, but also by establishing the infrastructure needed for project-based and experiential learning. Makerspaces have emerged as a key form of such infrastructure. These spaces combine collaborative workspaces, physical facilities, technical expertise and student communities, enabling students to turn conceptual ideas into tangible outcomes. Their importance in engineering design education lies in their ability to support fabrication, testing, failure analysis and iterative improvement in student-led projects [4]. In contrast to traditional teaching laboratories, makerspaces allow students to determine how different tools, materials and processes should be combined to address an open-ended engineering objective. Evidence from six university-based makerspaces shows that their educational contribution extends beyond equipment access: participation can support technical knowledge, collaboration, communication, creative problem solving, confidence, and the development of students' identities as engineers [5]. Academic makerspaces can therefore be understood as institutional learning infrastructures through which practice-oriented engineering education becomes operational.

The Tam Wing Fan Innovation Wing at The University of Hong Kong exemplifies this institutional model. The approximately 2,400 m² facility integrates assembly and brainstorming areas, specialised workshops, and prototyping technologies including three-dimensional printers, laser cutting and engraving machines, a waterjet cutting machine, electronic workbenches, computer-aided design facilities, and hand and power tools [6]. It also supports thematic workshops, interdisciplinary collaboration,

student-led projects, knowledge exchange and peer learning. By bringing resources, such as physical space, fabrication technologies and technical support, the Innovation Wing enables students to transform ideas developed in courses and independent projects into prototypes that can be constructed, tested, evaluated, and refined. Its educational value therefore lies not simply in the number of available machines, but in the range of engineering activities that those resources make possible.

The central challenge, however, is no longer simply whether suitable resources are available. The key lies in whether students with limited project experience can effectively access these resources. According to one study on engineering makerspaces, meaningful access entails more than just equipment availability or financial affordability; it also requires clear entry points and structured pathways that assist students in determining where to begin, which facilities are appropriate, and what procedures must be completed [7]. This method of access requires a high level of understanding. To participate effectively in makerspaces, you may need to grasp the equipment capabilities, manufacturing limits, materials, and other details. A mixed-method study of 151 undergraduate STEM students discovered that ambiguity about standards and feeling intimidated by more experienced makerspace users were important barriers to beginning involvement. [8]. Ethnographic research similarly shows that students become confident makers by gradually acquiring makerspace-specific vocabulary, procedural knowledge, and tacit “rules of the game” through interaction and participation [9]. These findings reveal a key distinction between

resource provision and resource usability: a makerspace may offer extensive technical capabilities while still existing entry barrier for new users.

Providing access to makerspace resources does not necessarily mean that students can effectively identify and utilise these resources for their projects. This challenge is particularly relevant in the HKU Innovation Wing. For example, a student seeking to fabricate an enclosure may need to choose among 3D printing, laser cutting, waterjet cutting or another manufacturing methods. Making this decision requires the student to identify the relevant technical requirements, compare suitable processes and understand the associated material. But in reality, these necessary information may be distributed separately and not straightforward. The resulting problem is the absence of a direct and accessible pathway that connects a student's project intention with the appropriate combination of resources and support.

These observations highlight a resource-navigation gap between students' project objectives and the distributed resources within an academic makerspace.

1.2 Motivation

Although the Innovation Wing provides students with access to a wide range of facilities, equipment, workshops, and technical expertise, the educational value of these resources depends on whether students can discover, understand and use them effectively. Three related difficulties may arise during this process. First, students may not have a complete understanding of what resources are available and may need to

search across webpages, equipment descriptions, workshop information or consult relevant staff before identifying a suitable facility. Second, even when students are aware of the available resources, they may not know which of them can help address the specific problem encountered in their project. Third, after identifying a potentially relevant resource, students may still be unfamiliar with its capabilities, access requirements, operating procedures or appropriate use. These difficulties create a gap between the provision of makerspace resources and their effective use by students. If students are unable to navigate this gap, the facilities provided to support project-based learning may be underutilised, used inefficiently, or accessed only by students who already possess sufficient technical knowledge and familiarity with the makerspace environment.

At present, students can obtain support through several channels, including the Innovation Wing website, equipment instructions, workshops, technical officers and other members of the makerspace community. These forms of support are essential because tutors and technical staff possess contextual knowledge, practical experience and professional judgement that cannot be fully replaced by static information sources. However, relying primarily on these channels for initial enquiries also presents several practical limitations. Tutors and technical staff may not always be available to provide immediate one-to-one assistance to every student, particularly during periods of high demand. A large proportion of their time may also be devoted to answering recurring questions concerning equipment availability, access procedures, workshop

arrangements and so on. In addition, students may be uncertain about which member of staff should be approached for a particular problem. Relevant information may be distributed across different webpages, documents, equipment instructions and staff roles, making it difficult for inexperienced users to determine where to begin. The timeliness of support may therefore depend on staff availability and whether the student can identify the correct source of assistance.

A more fundamental difficulty is that students generally approach the Innovation Wing from the perspective of a project objective rather than from the perspective of a particular facility. A student may wish to fabricate an enclosure, develop an interactive prototype, repair a mechanical component, or evaluate whether a design can be manufactured before a deadline. However, the student may not know whether the objective requires which part of resources. The resources of the Innovation Wing are necessarily organised according to facilities, equipment, services, activities, and operational responsibilities, but students commonly describe their needs in terms of the result they hope to achieve. A process is therefore required to translate an informal and potentially incomplete project objective into a set of technical requirements, identify the resources that may satisfy those requirements, and explain the actions needed to access and use them appropriately.

Motivated by this gap, the present study explores a more accessible and consistent form of digital support for students using the Innovation Wing. The proposed support is intended to allow students to express their project needs in their own words, discover

potentially relevant facilities and services, and understand the subsequent steps required to proceed. It is conceptualized as a mediating interface between students and the distributed resources within the Innovation Wing. Its purpose is to provide an initial point of guidance rather than to replace existing webpages, workshops, tutors and technical officers. In particular, it is not intended to replace mandatory safety training, professional technical judgement, staff authorisation, or direct supervision where these are required.

1.3 Outline

The remainder of this dissertation is organised as follows. Chapter 2 reviews the literature related to digital tutoring systems, large language models, tool use, task planning, and multi-agent systems, and identifies the research gap addressed in this study. Chapter 3 presents the methodology, a proposed digital tutor system with multi-layer frame. Chapter 4 describes the experimental settings and presents the evaluation results, including comparisons with the baseline system and analysis of system performance. Chapter 5 concludes this project.

Chapter 2 Literature Review

2.1 LLM-Based Digital Tutors in Education

Existing research already developed different kinds of digital tutor in various situation. One example is to supplement the support traditionally provided by lecturers and teaching assistants. These digital tutors' roles extend beyond answering subject-specific questions. They can also facilitate communication among different participants in the educational environment. Hobert and Berens argue that in large-scale courses, the high student-to-faculty ratio limits opportunities for individual interaction and feedback. Digital tutors can act as a bridge between students, teaching assistants and lecturers [10]. In this design, students interact with digital tutors in a simulated one-on-one environment, while teaching assistants and lecturers can access information about students' learning progress and difficulties. Therefore, this study positions digital tutors as an additional communication channel, instead of replacing human teachers.

Compared to previous systems that primarily relied on predefined rules or narrow student models, agents based on large language models(LLMs) can understand open-ended questions, maintain dialogue context and support a wider range of educational activities. Chu et al. categorized educational large-scale language model agents into instructional agents and domain-specific educational agents. Instructional agents provide various functions such as feedback and course-related tasks, while domain-specific agents are designed for specific disciplines or professional fields [11]. Their

research indicates that LLM agents have been applied to tasks for students and teachers, but it also highlights ongoing concerns about illusions, privacy, bias, over-reliance and integration with existing education systems. These limitations suggest that natural language fluency alone is insufficient to build reliable digital tutoring systems, especially when the system must provide institution-specific guidance.

One approach to improving the relevance and reliability of LLM-based educational support is to integrate feedback with local instructional materials. CourseAssist was developed specifically for computer science education. It combines retrieval-augmented generation(RAG), user intent classification and question decomposition to align generated feedback with course content and learning objectives [12]. Instead of allowing a general-purpose model to answer questions directly based on its pre-trained knowledge, this system retrieves relevant course materials and tailors its answers according to the type of help requested. CourseAssist illustrates how domain-specific knowledge retrieval and explicit request interpretation can improve the alignment of LLM-based educational support with a particular instructional context. But its main focus is on generating course-specific assistance, not coordinating multiple specialised for complex user requests.

Beyond responding to individual questions, LLM-based educational systems can also organize support around a learner's broader objective. GenMentor adopts a multi-agent framework for goal-oriented and personalised learning [13]. It identifies gaps between required skills and learners' existing skills and constructs appropriate learning paths.

Other components generate and integrate learning materials based on the identified needs. Its automated assessment, human evaluation, and deployment studies demonstrate that this framework can improve goal alignment, resource allocation, and personalization in professional learning environments. Unlike systems that only answer isolated questions, GenMentor uses learners' goals as a starting point for selecting and organizing subsequent resources. This goal-to-resource shift is conceptually relevant to makerspace users, who often start with desired project outcomes rather than prior knowledge of the facilities or processes needed to achieve their goals. Although this research also focuses on turning learners' objectives into real resources and materials, but it is not based on real makerspace environment and resources.

2.2 Task-Oriented LLM Agents and Task Decomposition

Being different with knowledge-oriented dialogue systems that primarily provide information or explanations, task-oriented dialogue systems are designed to help users accomplish specific goals through one or more conversational turns. This requires the system to identify the user's intent and extract relevant slot values. Hudeček and Dušek found that task-completion performance improved substantially when the model was provided with the correct task information and relevant in-domain examples [14]. This indicates that fluent language generation alone is insufficient for task-oriented support, as the system must also maintain reliable structured information about the user's objective. Furthermore, users may omit essential conditions because they lack technical knowledge or assume that the system understands their circumstances. ReSpAct

addresses this issue by allowing an agent to reason, act in an external environment, and communicate with the user to clarify ambiguous requirements, obtain preferences, and revise its plan [15]. Such clarification is particularly relevant to makerspace requests in which information such as the material, deadline, file availability, training status, or safety conditions may be required before an appropriate action can be selected.

After understanding the request, a task-oriented system must determine which specialist, resource, or operation should handle it. DARD uses a central dialogue manager to coordinate domain-specific agents, enabling requests to be delegated to independently developed specialist components and their responses to be integrated within a multi-domain dialogue system [16]. This architecture demonstrates the value of centralised routing, but routing alone mainly answers which agent should process a request and does not fully determine how a request involving several objectives should be completed. HuggingGPT extends this approach by treating an LLM as a controller that performs task planning, model selection, task execution, and response generation [17]. It decomposes a complex request into structured subtasks with explicit inputs and dependency relationships before selecting appropriate specialist models. Tasks without dependencies may be executed in parallel, while dependent tasks must be completed sequentially; more complex requests may form graph structures in which several intermediate results are combined by a later task. This explicit representation of dependencies is important when a single user request requires contributions from multiple tools or specialist agents rather than a single routing decision.

The relationship between planning and execution can also be organized in different ways. In ReAct, reasoning and tool use are interleaved, allowing each new observation to influence the next decision. This provides flexibility when intermediate results are uncertain, but repeated inference calls and the accumulation of previous observations increase token usage and execution costs. On the other hand, ReWOO breaks the process down into a planner, workers, and solvers. The planner first generates a blueprint of complete interdependent steps, the workers gather evidence through external tools and the solvers integrate the plan and evidence into the final response [18]. This separation can reduce model calls and improve efficiency, but its success largely depends on the accuracy and completeness of the initial plan. Therefore, these approaches represent a trade-off between adaptive execution and structured pre-planning: the former can respond more quickly to unexpected outcomes, while the latter provides a clearer and potentially more efficient execution structure.

Overall, existing task-oriented learning management systems provide important mechanisms for understanding intent, expert agent routing, tool usage and task decomposition. However, these functionalities do not directly meet the needs of digital mentors in makerspaces. Student requests may simultaneously involve in various resources in makerspace. Some tasks are independent, while others depend on previous results. Routing the entire request to a single domain agent may miss parts of the student's goals, while unrestricted task decomposition may result in unnecessary tasks, invalid dependencies, or inappropriate tool selection. Therefore, makerspace support

systems must not only understand and decompose requests but also differentiate between simple and complex requests, validate the resulting task structure, match tasks with available resources and tools, and ensure the full achievement of overall project goals.

2.3 Multi-Agent Orchestration and State-Driven Workflow

LLM-based multi-agent systems distribute different parts of a complex task among specialised agents, but the use of multiple agents alone does not guarantee reliable collaboration. Without explicit coordination, agents may perform duplicated work, generate inconsistent outputs, or propagate errors from one stage to the next. MetaGPT illustrates that role-specialised agents can be coordinated through predefined responsibilities, standard operating procedures, and structured intermediate outputs rather than unrestricted conversation [19]. More directly relevant to the present study, StateFlow represents LLM task-solving as a state-driven workflow in which each state corresponds to a particular stage of the process and transitions determine the next permitted action [20]. The shared state may record information collected from the user, generated plans, tool outputs, validation results, and error conditions, while individual states may invoke differently prompted LLMs or external tools. Separating process control from task execution makes the workflow easier to inspect and constrains which operations can occur at each stage. This is particularly relevant to a makerspace Digital Tutor, where request understanding, planning, verification, policy checking, and tool execution must exchange information and occur in a controlled order.

Finite-state representations have also been applied to the construction of multi-agent systems. MetaAgent automatically generates agent roles and collaboration structures from task descriptions and represents their interactions as a finite-state machine [21]. This demonstrates that states and conditional transitions can provide a formal structure for coordinating multiple agents. However, MetaAgent aims to construct and optimise an agent architecture automatically, whereas the workflow in the present study is manually designed according to the requirements of Innovation Wing support. Its relevance therefore lies primarily in demonstrating the broader applicability of finite-state models to multi-agent orchestration, rather than providing a directly adopted architecture.

Although state-driven workflows improve process control, one fixed workflow may still be inefficient when user requests differ substantially in complexity. Multi-Agent Architecture Search via Agentic Supernet selects query-specific combinations of reasoning operators, agents, tools, and model calls from a broader architecture space [22]. While this architecture-search method is considerably more general than the approach used in this study, it supports the principle that different queries may require different execution structures. Difficulty-Aware Agentic Orchestration develops this argument more directly by observing that static workflows can over-process simple queries while allocating insufficient reasoning capacity to difficult ones [23]. It estimates the difficulty of an input and adjusts workflow depth, agentic operations, and model allocation while considering both expected performance and computational cost.

This principle is particularly relevant to makerspace assistance: locating a facility may require only direct retrieval, whereas evaluating whether a model can be manufactured may require file analysis, time and material estimation, rule checking, and the integration of several dependent results.

Overall, the literature suggests that multi-agent orchestration should be applied selectively rather than assumed to be inherently superior to a direct or single-agent approach. Processing a simple request through a fixed multi-stage workflow may unnecessarily increase token consumption, API calls, latency, and potential failure points. Errors in request understanding or planning may also propagate through downstream components, while repeated communication between agents may omit details from the original request. In contrast, explicit state-driven workflows can provide clearer task coverage, execution traces, conditional transitions, dependency control, and intermediate validation when a request contains multiple interdependent operations. These findings support a complexity-aware architecture in which simple requests follow a lightweight execution path, while complex requests are decomposed and processed through a structured workflow. However, existing adaptive orchestration studies have primarily been evaluated on general reasoning, coding, mathematical, or tool-use benchmarks. They do not directly investigate how request complexity should be represented in a makerspace environment, where it may depend on the number of user objectives, the facilities and tools involved, and the parallel, sequential, or graph-based dependencies among project-support tasks.

2.4 Research Gap

Overall, existing studies demonstrate the value of structured workflows, state-based control, and query-specific orchestration in multi-agent systems. However, most have been evaluated on general reasoning, coding, mathematical, or tool-use benchmarks rather than engineering makerspace support. They do not explain how makerspace requests should be classified according to the number of objectives, required tools, and parallel, sequential, or graph-based dependencies. It also remains unclear whether structured multi-agent orchestration provides sufficient benefits over a simpler single-agent approach to justify its additional cost and complexity in this context.

Chapter 3 Methodology

3.1 System Requirements and Design Objectives

3.1.1 Application Requirements

The Digital Tutor is designed for common project-support scenarios within the Innovation Wing. These scenarios include searching for suitable facilities or equipment, identifying relevant workshops, obtaining information about access and operating requirements, and receiving guidance on three-dimensional printing tasks. For example, students may ask the system to recommend a facility for fabricating a component, identify a workshop that teaches a required skill, inspect a three-dimensional model, estimate its printing time and material consumption, check whether it complies with printing rules, or explain the subsequent submission procedure.

The system also supports requests expressed in terms of project. For example, a student may wish to fabricate an enclosure or complete a prototype before a deadline without knowing which manufacturing process or facility is appropriate. In this case, although the student hasn't specified what resources are needed, the system should interpret the intended outcome, identify the relevant project-support needs, and connect those needs with available resources or tools. In reality, some request contain single intent and some contain multiple intents.

3.1.2 Functional Requirements

To support the application scenarios, the digital tutor should convert open-ended natural language requests into structured, executable operations. Instead of interpreting each user input as a single predefined intent, it can determine one or more goals inside it. This enables the digital tutor to handle both simple information retrieval queries and requests for coordinated multi-step problem solutions.

The following major characteristic is adaptive task planning. For a single intent request, the system directly invokes the appropriate specialized tool, reducing additional planning time. When a request contains numerous intents, the digital tutor divides the goal into executable subtasks and models the relationships between them. Independent subtasks can run concurrently, but dependent subtasks are scheduled according to their prerequisites.

To resolve various request, a set of tools are developed based on real user request and knowing resources and services. Each subtask is mapped to a corresponding specialized tool and its execution parameters are constructed based on the user request and the dialogue context. Before execution, the digital tutor validates the generated process against available operations and dependency structures, ensuring internal consistency and executable capability.

Finally, the digital tutor generates final nature language response to answer user inquiry based on the execution results instead of presenting fragmented outputs from individual

tools.

3.1.3 Design Objectives

The proposed digital tutor system is designed based on following objectives:

Firstly, the system should lower the knowledge barrier to using makerspace resources.

Students should be able to start with an informal project objective communicated in common language rather than having to know the names, capabilities, or organizational structure of specific Innovation Wing facilities ahead of time.

Secondly, the system should be designed with a modular and traceable system architecture. Request understanding, task planning, tool execution, and response generation are implemented as distinct processing functions. This separation enables specialized capabilities and tools to be developed or upgraded independently while preserving intermediate system states that link the initial user request to the generated workflow and executed activities.

Thirdly, a structured orchestration according to the requirements of each request should be implemented. Simple enquiries should follow a lightweight execution pathway, while requests requiring multiple coordinated operations should use explicit task decomposition and dependency-aware execution. This allows the system to provide structured processing for complex requests without applying the same orchestration process to every interaction.

3.2 Overall System Architecture

The proposed digital tutor is organised as a state-based workflow consisting of four main components: the understanding and control Layer, the planning layer, the execution layer, and the response-generation component. A user request first enters the Understanding Layer and Control Layer, which identifies the relevant intents and determines whether the request should follow a simple or complex path. Simple requests are directly mapped to the tool selection stage, while complex requests are forwarded to the Planning Layer for task decomposition and dependency planning. The resulting tasks are then passed to the Execution Layer, which prepares the required tool inputs. Finally, the response-generation component integrates the execution results and produces a natural-language response aligned with the student's original objective.

The components are connected through a shared state managed by LangGraph. This state records information such as the original request, identified intents, request type, generated plan, task dependencies, execution results, and final response, allowing outputs from one stage to be reused by subsequent stages. Figure 3.1 illustrates the overall architecture and the flow of information between the main system modules.

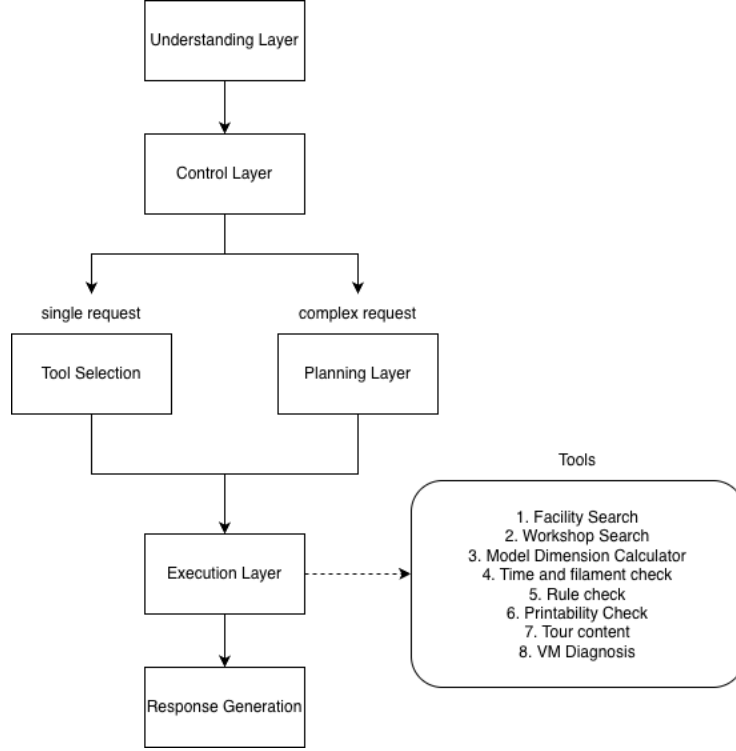


Figure 1 Proposed System Architecture

3.3 Request Understanding and Control Layer

The Understanding Layer converts an unstructured user query into a structured representation consisting of a set of intents and a collection of task-relevant entities.

Formally, given a user query q , the layer produces

$$U(q) = (I)$$

where $I = \{i_1, \dots, i_m\}$ denotes the detected intents. The intent space is restricted to seven domain-specific categories: general information, workshop inquiry, print-submission checking, equipment recommendation, printability checking, troubleshooting, and tour requests.

The layer is implemented using a LLM with structured prompting. The prompt specifies

the permitted intents, the entity schema, intent-selection rules, and few-shot. To reduce prompt length while retaining task relevance, the system computes the semantic similarity between the input query and the available demonstrations and selects the eight most relevant examples. The model output is constrained to a predefined JSON schema and subsequently validated using a typed data model. For multimodal requests, metadata extracted from attached images or 3D-model files is incorporated into the structured representation before routing.

The Control Layer determines whether the structured request can be handled through a single execution path or task decomposition and planning. It operates on the normalized intent set produced by the Understanding Layer:

$$C(I) = \begin{cases} \textit{Single}, & |I| \leq 1, \\ \textit{Complex}, & |I| > 1, \end{cases}$$

This deterministic routing policy separates semantic interpretation from workflow control. The Understanding Layer identifies what the user requires, while the Controller Layer determines how the request should be processed. This separation improves interpretability and allows for checking whether intent recognition errors occur early in the system's development.

3.4 Planning Layer

3.4.1 Task Decomposition and Dependency Planning

The Planning Layer decomposes a complex user request into an executable task dependency graph. It receives the original request q , the intent set I produced by the

Understanding Layer, and the available contextual information X . The planning operation can be expressed as

$$P(q, I, X, \mathcal{R}) \rightarrow (s, T),$$

where $\mathcal{R}$ is the task registry, s is the planning status, and T is the generated task set.

The planning status is one of ready, needs_clarification, or cannot_plan.

Each planned task is represented as:

$$t_i = (id_i, a_i, D_i, \theta_i, r_i),$$

where id_i is the task identifier, a_i is a task selected from the registry, D_i is the set of predecessor tasks, θ_i contains the task arguments, and r_i provides a short explanation of the dependency relationship. Task selection to a registry of pre-defined tasks decreases the possibility of unsupported or non-executable operations.

Prior to invoking the LLM, the system filters the entire task registry according to the detected intents and the predefined intent-to-tool mappings. This way the planner receives only tasks relevant to the current request. The prompt also includes examples of specific intents to show how to properly decompose tasks and build dependencies. The model should output a JSON structured object following a predefined pattern.

3.4.2 Dependency Graph Construction

A ready plan is represented as a directed acyclic graph (DAG)

$$G = (V, E),$$

where each vertex $v_i \in V$ corresponds to a planned task. An edge

$$(v_j, v_i) \in E$$

indicates that task t_i depends on the output of task t_j and therefore cannot be executed before t_j has completed. Tasks without dependency edges can be executed in parallel.

When a downstream task requires a value produced by an upstream task, the planner uses an explicit resource reference of the form

$$< resource > - t_j.field$$

For example, a printer-recommendation task may consume a material or model dimension obtained from an earlier model-analysis task. This explicit representation allows the Execution Layer to resolve intermediate results without requiring the language model to rewrite them into subsequent prompts.

3.4.3 Plan Validation

Because language-model-generated plans may contain structurally valid JSON while remaining operationally invalid, every plan is checked by a deterministic validator before execution. The validator verifies following aspects:

1. task identifiers are unique and sequential,
2. every task belongs to the permitted task registry,
3. every detected intent is covered by at least one planned task,

4. all required task inputs are available from the query context, task arguments, or upstream outputs,
5. dependencies refer only to existing earlier tasks,
6. the dependency graph contains no self-dependencies or cycles,
7. each resource reference is supported by an explicit dependency and refers to a field produced by the upstream task.

A plan is forwarded to the Execution Layer only when its status is `ready` and all deterministic validation checks have passed. If there is missing critical information, the planner updates the status to `needs_clarification` and notes the missing inputs and an appropriate clarification question in the shared state. The incomplete plan is not handed over for execution.

3.4.4 Plan-structure Classification

After validation, the task graph is deterministically classified into one of four executable structures:

$$S(G) = \begin{cases} \text{single,} & |V| = 1, \\ \text{parallel,} & |V| > 1 \wedge E = \emptyset, \\ \text{sequential,} & G \text{ forms a dependency chain,} \\ \text{graph,} & \text{otherwise.} \end{cases}$$

Parallel plans contain several independent tasks. Sequential plans have tight dependency chains where each task depends solely on the previous task. The class of graph-structured plans contains all other valid DAGs.

This classification gives explicit scheduling information to the execution layer yet

preserves semantic decomposition and separation of runtime scheduling components.

3.5 Execution Layer

The execution layer calls the domain tools in a consistent dependency order according to the validated task graph produced by the planning layer and collects structured evidence to generate the response.

For individual requests, the selected tool is invoked through a shared tool adapter that normalizes the tool name and parameters. If the structured result is successful, it is forwarded to the response generation layer. Else the request is directed to a fallback path with explicit error logging.

Execution for a complex request starts only when the plan is ready and has passed plan validation. Let P, C, F and $\mathbf{B}$ denote the pending, completed, failed, and blocked task sets. At each scheduling iteration, the executor identifies the ready tasks

$$R = \{t_i \in P \mid D_i \subseteq C\},$$

where D_i is the dependency set of task t_i . Tasks in R are logically independent at that point.

Before each task runs, the Resource Registry updates the intermediate-result references created by the planner with actual values obtained from completed upstream tasks. It also ensures a consistent set of output fields is maintained between known pairs of tools.

When explicit task arguments are provided, they override any automatically projected values.

Table 1 Task-to-Tool Mapping and Main Outputs

Planned task	Tool or component	Main output
facility_search	Facility Search Tool	Relevant facility records
workshop_search	Workshop Search Tool	Relevant workshop information
model_dimension_calculator	Model Analysis Tool	Model dimensions
time_and_filament_check	Printing Estimation Tool	Time and filament
rule_check	Rule Check Tool	Compliance result
printability_check	Printability Component	Overall decision

Each ready task is invoked through the Tool Adaptor. The returned value is checked to ensure that it is structured, contains no explicit error, and provides either an expected output field or a recognized success indicator. Successful outputs are stored by task identifier and become available to downstream tasks.

If a task fails, dependent tasks are marked as blocked and are not executed. Independent branches may continue. The final graph-execution status is defined as

$$Z(G) = \begin{cases} \text{completed,} & |C| = |V|, \\ \text{partially completed,} & |C| > 0 \wedge (|F| + |B|) > 0, \\ \text{failed,} & |C| = 0. \end{cases}$$

Completed and partially completed results are forwarded to response generation, together with per-task execution records. A fully failed execution is routed to the fallback response path.

3.6 Response Generation

3.6.1 Evidence Preparation

The Response Generation Layer transforms structured execution results into a concise and user-facing answer. Rather than allowing the language model to answer directly from the original request, this layer requires the response to be grounded in evidence produced by successfully executed tools. Let q denote the original user request and let $\mathcal{E}$ denote the structured evidence obtained from the Execution Layer. The response-generation process is expressed as

$$y = L(q, \Pi(\mathcal{E}))$$

where L is the response-generation language model, Π is a deterministic evidence-projection function, and y is the final response. The layer consists of three stages: evidence preparation, grounded response synthesis, and output validation with fallback handling.

3.6.2 Grounded Response Synthesis

The predicted evidence and the original user request are provided to the response generation language model. This model is instructed to use execution evidence as the

sole source of task-specific facts. It must directly respond to the original objective, covering all successfully completed parts of the request, retaining relevant numerical values and units, and clearly reporting failed or blocked operations.

Chapter 4 Experiments and Evaluation

4.1 Research Questions

This study evaluates the proposed Digital Tutor at both the module and system levels. The module-level evaluation examines the Request Understanding and Control pipeline, the Planning Layer, and the Execution Layer under controlled upstream inputs. This design allows the performance of each evaluated module to be distinguished from errors originating in earlier stages. The end-to-end evaluation examines the complete workflow, including response generation, under realistic error propagation, where an error made by an earlier module may affect planning, tool execution, and the final user-facing response. The experiments are organised around the following research questions.

RQ1: How accurately does the Request Understanding and Control pipeline identify user intents and route requests to the appropriate processing path?

This question evaluates whether the system can transform an unstructured user request into the intended set of domain-specific intents and correctly determine whether the request should follow the direct execution path or be forwarded to the Planning Layer. Because the Controller applies a deterministic routing rule to the intents predicted by the Understanding Layer, intent recognition and request routing are evaluated as a combined pipeline rather than attributing routing performance to the Controller alone. Performance is measured using intent-set exact match, micro-averaged precision, recall

and F1-score, routing accuracy, and a routing confusion matrix.

RQ2: How accurately does the Planning Layer transform complex requests into valid and executable task dependency graphs?

This question examines whether the planner selects the required tasks, identifies the correct dependencies among them, and recovers the intended execution structure. Parallel, sequential, and general graph-structured requests are evaluated separately because they require different forms of dependency reasoning. To isolate planning performance from upstream intent-recognition errors, the module-level planning experiment supplies the planner with ground-truth intents.

Planning quality is measured using task-set precision, recall, F1-score and exact match; dependency-edge precision, recall and F1-score; dependency-graph exact match; plan-structure accuracy; and validation pass rate. These measures evaluate both the semantic correctness of the selected tasks and the structural correctness of the generated dependency graph.

RQ3: How reliably can the Execution Layer carry out validated task graphs and preserve structured evidence for subsequent response generation?

This question investigates whether the executor can invoke the required tools in dependency-consistent order, resolve intermediate results between tasks, and prevent downstream execution when an upstream dependency fails. The module-level execution experiment uses ground-truth task graphs so that execution failures can be

distinguished from planning errors.

The evaluation measures tool-invocation success, request-level execution success, intermediate-result resolution, failed and blocked tasks, and categorized execution errors. It also investigates whether independent branches can persist when one branch fails and if successfully completed outputs are maintained as organized evidence. The quality of the final response for users is not assessed separately at the module level but is evaluated as a component of the overall system assessment.

RQ4: How does the proposed modular system compare with simpler tool-using baselines in effectiveness and efficiency?

This question evaluates whether separating request understanding, task planning, tool execution, and response generation provides measurable benefits over simpler tool-using baseline systems. The comparison considers overall task resolution, mandatory-requirement coverage, response completeness, groundedness, required-tool recall, unnecessary-tool use, tool-order accuracy, and tool-call success.

The results are also analyzed by request structure to determine whether the relative benefit of modular orchestration changes across single, parallel, sequential, and graph-structured requests. In addition to effectiveness, response latency, token consumption, and the number of language-model calls are measured to quantify the computational overhead introduced by the modular architecture.

Table 2 Summary of Research Questions and Evaluation Measures

Research question	Evaluation scope	Principal measures
RQ1	Understanding–Control pipeline	Intent-set exact match, micro-F1, routing accuracy
RQ2	Planning Layer	Task-set F1, graph exact match, edge-F1, structure accuracy
RQ3	Execution Layer	Invocation success, request-level success, intermediate-result resolution, failed and blocked tasks
RQ4	End-to-end system and baselines	Resolution, requirement coverage, response completeness, groundedness, tool use, latency, and tokens

4.2 Evaluation Dataset

A purpose-built evaluation dataset was constructed to assess the proposed Digital Tutor at the module and system levels. The dataset contains 100 user requests situated in the HKU Innovation Wing domain and covers facility and equipment enquiries, workshop retrieval, equipment recommendation, 3D-printing submission checks, model printability assessment, visual troubleshooting, and tour-content generation.

The dataset includes text-only and multimodal requests in both English and Chinese. Image-based cases contain photographs used for manufacturing-failure diagnosis, while model-based cases contain 3D-model files used for dimension extraction, print-time and filament estimation, rule checking, and printability assessment.

Each case contains the following annotations:

$$d_i = (q_i, r_i, I_i, S_i, G_i),$$

where q_i is the user request, r_i is the expected request type, I_i is the set of expected intents, S_i is the expected execution structure, and G_i is the annotated task graph. Each task in G_i specifies a task identifier, an executable tool name, and a set of predecessor dependencies.

The annotations therefore support evaluation at multiple levels. Expected intents and request types are used to evaluate the Understanding–Control pipeline. Expected task sets and dependency edges are used to evaluate the Planning Layer. The annotated task graphs also provide controlled inputs for the isolated Execution Layer evaluation. Finally, the original user requests and their requirements are used in the end-to-end assessment.

The dataset contains 40 single requests and 60 complex requests. The assessment requests and their corresponding annotations were first created and then manually examined and modified. Every case was reviewed to confirm that the anticipated intents, necessary tasks, and dependency connections aligned with the functionalities integrated into the Digital Tutor.

4.3 Experimental Design

The experimental setup integrates module-level assessment with comprehensive system comparison. Experiments at the module level examine the performance of single components by providing accurate outputs from earlier layers. End-to-end experiments present solely the initial user request and its accompanying files, enabling errors to

spread throughout the entire workflow.

4.3.1 Compared Systems and Baselines

Three tool-using systems were included in the comparison: the proposed modular system, a pure monolithic baseline, and an iterative monolithic baseline. All systems were provided with the same domain tools, task registry, knowledge base, attachment-processing utilities, LLM and evaluation requests. The difference among them is how they organize reasoning, tool selection, dependency handling, and response generation.

Proposed modular system

The proposed system separates the processing workflow into the following components:

1. Understanding Layer;
2. Control Layer;
3. Tool Selection;
4. Dependency-aware Planning;
5. Execution Layer
6. Response Generation.

The Understanding Layer extracts intents and entities from the request. Multi-intent requests are forwarded directly to the Planning Layer, while single-intent requests first enter Tool Selection. A single-intent request may still be forwarded to planning when the selected capability requires multiple atomic operations.

For planned requests, the system constructs and validates a task dependency graph

before execution. Intermediate tool outputs are passed through explicit resource references, and the final answer is generated from structured execution evidence.

Pure Monolithic Baseline

The pure monolithic baseline uses one language-model agent without separate understanding, routing, planning, validation, dependency-resolution, or response-generation components.

The agent operates in two model calls. In the first call, it selects one batch of tools that can run independently from the original request. The selected tools are then executed, and their observations are returned to the same agent in a second call for final-response generation.

The agent has only one opportunity to select tools. It cannot issue a dependent tool call after examining an upstream result, construct a multi-stage dependency graph, replan, or recover through an additional tool-selection step. A maximum of eight tools may be selected in the initial batch.

This baseline tests whether explicit planning and dependency-aware orchestration provide benefits beyond a conventional select-tools-then-answer workflow.

Iterative Monolithic Baseline

The iterative monolithic baseline also uses a single language-model agent but permits repeated reasoning and tool-use steps. At each step, the agent either produces a batch task graph or returns a final answer. Within a batch, it may specify task dependencies

and intermediate-result references. The executor runs ready tasks and returns their observations to the same agent.

Unlike the proposed system, this baseline does not contain independently evaluated Understanding, Control, Planning, Plan Validation, Execution, or Response Generation layers. Task interpretation, tool selection, dependency construction, recovery, and final-answer generation are controlled by a single agent prompt.

The iterative baseline is allowed up to 12 decision steps. It therefore represents a stronger monolithic agent than the pure baseline and is used to examine whether explicit modular separation offers benefits beyond allowing a single agent to reason iteratively.

4.3.2 Implementation Settings

All experiments were implemented in Python using LangGraph for workflow orchestration, Pydantic for structured-output validation, Chroma for vector retrieval, and the OpenAI-compatible model interface for language-model access.

The principal language model used in the current experiments was GPT-5.4, accessed through Azure OpenAI using API version 2024-12-01-preview. The same deployment was used for text generation and multimodal image analysis. The maximum completion-token allowance was set to 16,384 tokens.

The Azure execution path did not explicitly specify a sampling temperature. Therefore, the experiments used the deployment’s supported/default decoding behaviour rather than claiming deterministic temperature-zero decoding. All model outputs used for

intent extraction, planning, tool selection, and evaluation judging were requested in structured JSON form where applicable.

The Understanding Layer used the intfloat/multilingual-e5-small sentence-transformer model to select in-context demonstrations. For each request, the eight demonstrations with the highest semantic similarity were included in the Understanding prompt. Embeddings were normalized before similarity calculation.

Facility retrieval used the local Chroma index and the same multilingual embedding model. All compared systems accessed the same facility records, workshop data, printing rules, tour content, tools, and attachment files. Therefore, differences in results are attributable primarily to orchestration and prompting rather than differences in available domain information.

The module-level execution experiment applied a 30-second timeout to each individual tool invocation. For the final end-to-end comparison, the same request-level timeout should be applied to all three systems. A value of 180 seconds is recommended because it accommodates the iterative baseline’s additional reasoning steps while still bounding unsuccessful executions.

Table 3 shows settings that used for the final comparison.

Table 3 Settings for final comparison

Setting	Value
Main model	GPT-5.4
Model provider	Azure OpenAI

Setting	Value
API version	2024-12-01-preview
Maximum completion tokens	16,384
Understanding demonstrations	Top 8 by semantic similarity
Embedding model	intfloat/multilingual-e5-small
End-to-end timeout	180 seconds per request
Execution module timeout	30 seconds per tool invocation
Pure monolithic tool batches	1
Pure monolithic maximum tools	8
Iterative monolithic maximum steps	12
Evaluation cases	100

4.3.3 Evaluation Metrics

The evaluation uses separate metrics for request understanding, planning, execution, final-response quality, tool use, and efficiency.

Understanding Metrics

Let I_i and $\hat{I}_i$ denote the expected and predicted intent sets for request i .

Exact intent-set match measures whether the complete predicted set is identical to the annotation:

$$\text{ExactMatch} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{I}_i = I_i).$$

Intent precision and recall are micro-averaged across all cases:

$$\begin{aligned}
P_{\text{intent}} &= \frac{\sum_i TP_i}{\sum_i (TP_i + FP_i)}, \\
R_{\text{intent}} &= \frac{\sum_i TP_i}{\sum_i (TP_i + FN_i)}, \\
F1_{\text{intent}} &= \frac{2P_{\text{intent}}R_{\text{intent}}}{P_{\text{intent}} + R_{\text{intent}}}.
\end{aligned}$$

For Controller evaluation, the expected class is derived from the annotated intent set:

$$r_i = \begin{cases} \text{single-intent,} & |I_i| = 1, \\ \text{multi-intent,} & |I_i| > 1. \end{cases}$$

Routing performance is reported using accuracy, precision, recall and F1 for the multi-intent class, together with a confusion matrix.

Planning Metrics

Planning is evaluated using both task selection and dependency construction.

Let A_i and $\hat{A}_i$ denote the multisets of expected and predicted executable tasks. Task-set precision, recall and F1 measure whether the planner selects the correct tools. Task-set exact match measures the proportion of cases for which the predicted and expected task multisets are identical.

Dependency evaluation compares directed, task-labelled edges rather than generated task identifiers. An expected dependency from task type a to task type b is represented as (a, b). This prevents semantically equivalent plans from being marked incorrect merely because they use different task IDs.

Dependency accuracy is the proportion of cases for which the complete predicted dependency-edge multiset matches the annotation:

$$\text{DependencyAccuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{E}_i = E_i).$$

Dependency-edge precision, recall and F1 are also reported across all annotated edges.

Plan-structure accuracy measures whether the predicted plan is correctly classified as single, parallel, sequential, or general graph. Validation pass rate reports the proportion of generated plans accepted by the deterministic validator.

For parallel plans, both expected and predicted edge sets may be empty. These cases contribute to dependency accuracy but not meaningful edge-level precision or recall.

Execution Metrics

The isolated Execution Layer experiment reports:

- planned tool invocations;
- successful tool invocations;
- tool-invocation success rate;
- request-level execution success;
- failed invocations;
- blocked invocations;
- execution errors by category.

Tool-invocation success rate is defined as

$$\text{ToolSuccess} = \frac{N_{\text{successful calls}}}{N_{\text{planned calls}}}.$$

A request is successful when all planned tasks complete and partially successful when

at least one task completes and at least one.

Errors are categorized as tool-not-found, input-adaptation, file-access, runtime, timeout, output-schema, or dependency errors.

Final-response Metrics

End-to-end responses are evaluated by a separate language-model judge. The judge receives:

- the original user request;
- expected intents;
- expected tasks;
- the final response;
- structured execution evidence.

Each response receives a completion score:

$$s_i \in \{0, 1, 2\},$$

where 0 indicates failure, 1 indicates partial completion, and 2 indicates full completion.

Mean completeness is normalized to the range $[0,1]$:

$$\text{MeanCompleteness} = \frac{\sum_{i=1}^N s_i}{2N}.$$

Task-resolution rate measures the proportion of requests for which the core user goal is judged to have been solved:

$$\text{TaskResolution} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{core task resolved}_i).$$

Mandatory-requirement coverage is the mean fraction of explicit required elements substantively addressed by the response.

Groundedness pass rate measures the proportion of responses whose material factual claims and recommendations are supported by execution evidence:

$$\text{Groundedness} = \frac{N_{\text{grounded responses}}}{N}.$$

A response fails groundedness when it materially contradicts tool evidence, invents a facility, workshop or tool result, or presents an unsupported definitive claim.

The judge also records abstentions. Acceptable-abstention rate measures the proportion of abstaining responses that correctly explain the unavailable result, limit the scope of the refusal, and provide an appropriate next step.

Tool-use Metrics

Required-tool recall measures the proportion of annotated tool invocations that appear in the actual execution trace:

$$\text{RequiredToolRecall} = \frac{\sum_i |A_i \cap \hat{A}_i|}{\sum_i |A_i|}.$$

The calculation uses multisets so that repeated uses of the same tool are counted separately.

Unnecessary-tool rate measures the proportion of actual calls that do not correspond to

an expected tool invocation:

$$\text{UnnecessaryToolRate} = \frac{\sum_i |\hat{A}_i - A_i|}{\sum_i |\hat{A}_i|}.$$

Tool-order accuracy measures the proportion of annotated dependency edges respected by the observed execution order:

$$\text{ToolOrderAccuracy} = \frac{N_{\text{dependency edges respected}}}{N_{\text{annotated dependency edges}}}.$$

Tool-call success rate is the proportion of actual tool calls that complete successfully:

$$\text{ToolCallSuccess} = \frac{N_{\text{successful actual calls}}}{N_{\text{actual calls}}}.$$

Required-tool recall measures coverage, while tool-call success measures runtime reliability. A system may therefore achieve high call success while still omitting a required tool.

4.4 Module-Level Evaluation Results

This section evaluates the major modules of the proposed system separately in order to understand how each component contributes to the overall system performance.

4.4.1 Understanding and Control Layer

The request understanding module is responsible for identifying the user's intent from the original query.

For intent recognition, the system achieved an exact-match accuracy of 94.0%. In terms of micro-averaged intent classification, the module obtained a precision of 96.53%, a

recall of 98.24%, and an F1 score of 97.38%. These results indicate that the understanding module was generally able to recover the correct user intents, with a slightly higher recall than precision. This means that the system was more likely to include the required intent than to miss it, although a small number of additional intents were occasionally introduced.

For the control decision, the expected routing label was derived from the number of annotated expected intents: requests with one intent were treated as single-intent requests, while requests with more than one intent were treated as complex requests. Under this definition, the controller achieved an accuracy of 97.0%. The complex-request class obtained a precision of 96.36%, a recall of 98.15%, and an F1 score of 97.25%. Only three routing errors were observed across the 100 evaluation cases.

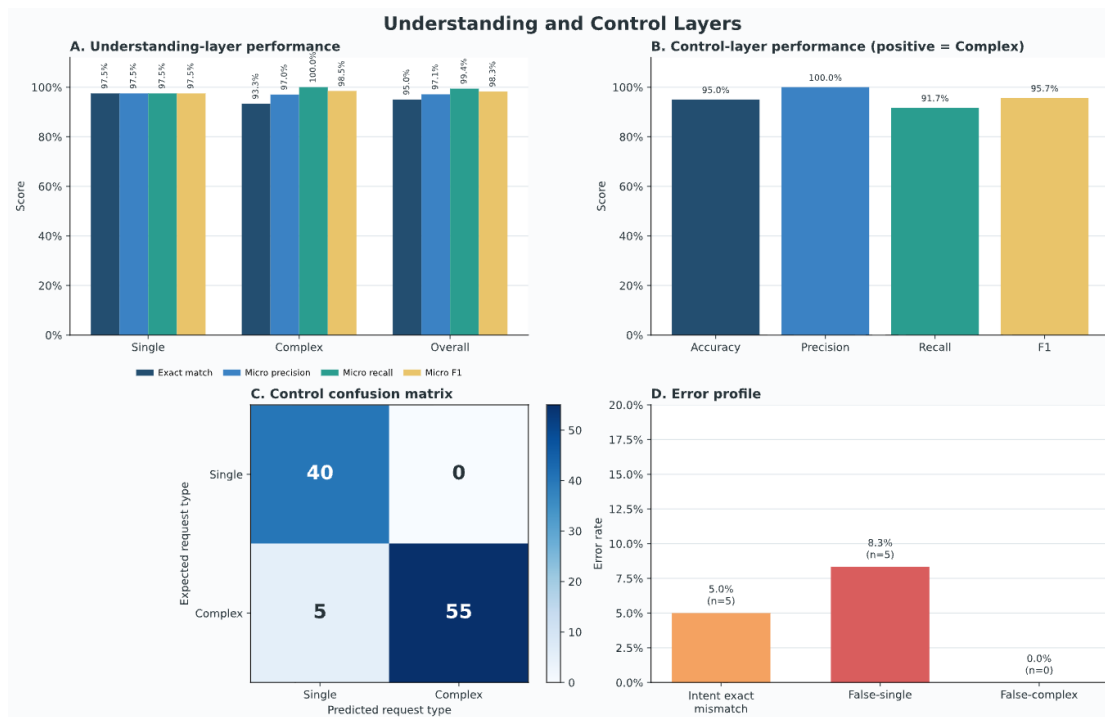


Figure 2 Performance of the Understanding and Control Layers

The main source of routing error came from intent boundary ambiguity. For example, in some model-related queries, the system predicted both `print_submission_check` and `printability_check`, while the annotation only required one of them. This led to a single-intent request being routed as complex. In another case, the system missed one intent in a genuinely complex request, causing it to be routed as a single-intent request. These errors show that the control module itself is relatively reliable, but its correctness depends strongly on the granularity and completeness of the understanding module's intent prediction.

Overall, the results show that the Understanding and Control pipeline achieved high intent-recognition and routing accuracy, although routing remained sensitive to missing or additional intent predictions.

4.4.2 Planning Layer

The planning module was evaluated on the complex planning subset, where each case requires the system to decompose a user request into one or more tool-based tasks and specify the dependencies among them. To isolate planning ability from understanding errors, the planner was evaluated using the gold expected intents as input. It is used to determine if the planner is able to create an appropriate task structure in the case where the user's intent has already been properly identified.

The planner had a 100.0% in tool-set precision, 97.4% in tool-set recall, and an 98.7% in toolset F1. The exact task-set match was 95.0%. The perfect precision indicates that

the planner almost never selected unnecessary tools in the isolated setting. Most planning errors came from missing required tasks rather than adding irrelevant ones.

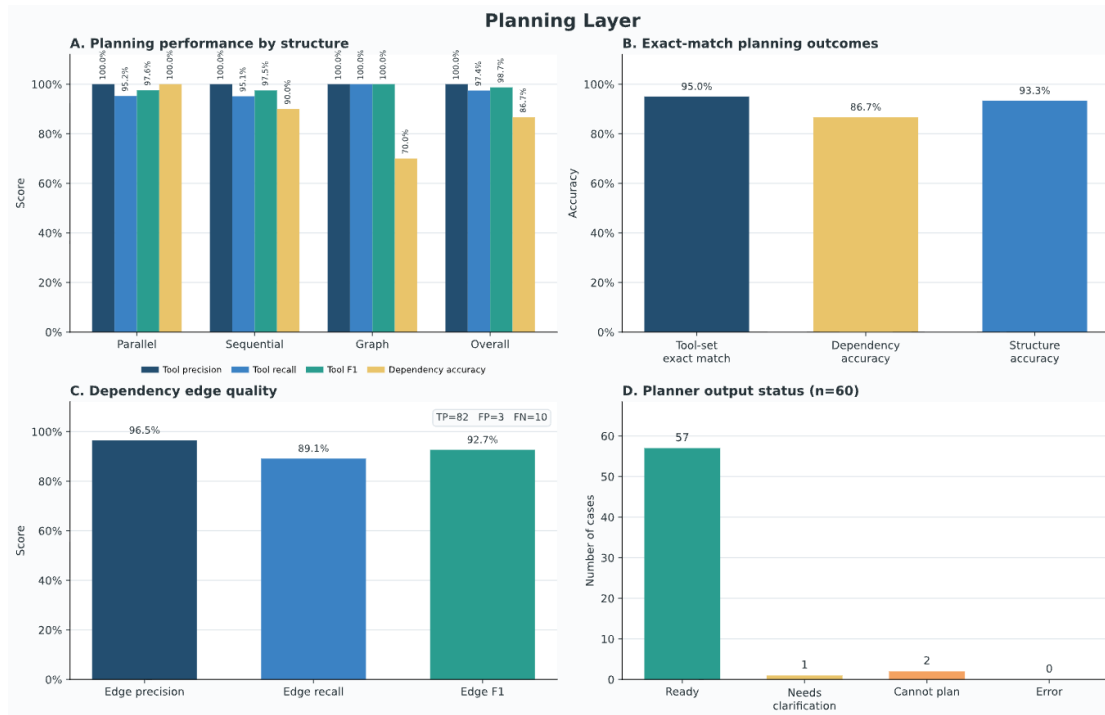


Figure 3 Performance of the planning layer.

The planner achieved 86.7% in dependency accuracy, 96.5% in edge precision, 89.1% recall and 92.7% F1 score. These data show that the dependency edges generated by the planner are generally correct, but sometimes necessary dependencies are missed.

Performance also varies depending on different types of complex request structure. For parallel and sequential requests, dependency accuracy individually reached 100.0% and 90.0%, indicating that the planner handles independent request and simple linear dependencies well. Graph structure requests are relative challenging, with dependency accuracy dropping to 70.0%. The planner achieved 93.33% in classification different

complex request structure.

These results indicate that the planning layer works well for tool selection and simple dependency structures but graph based requests are still the hardest.

4.4.3 Execution Layer

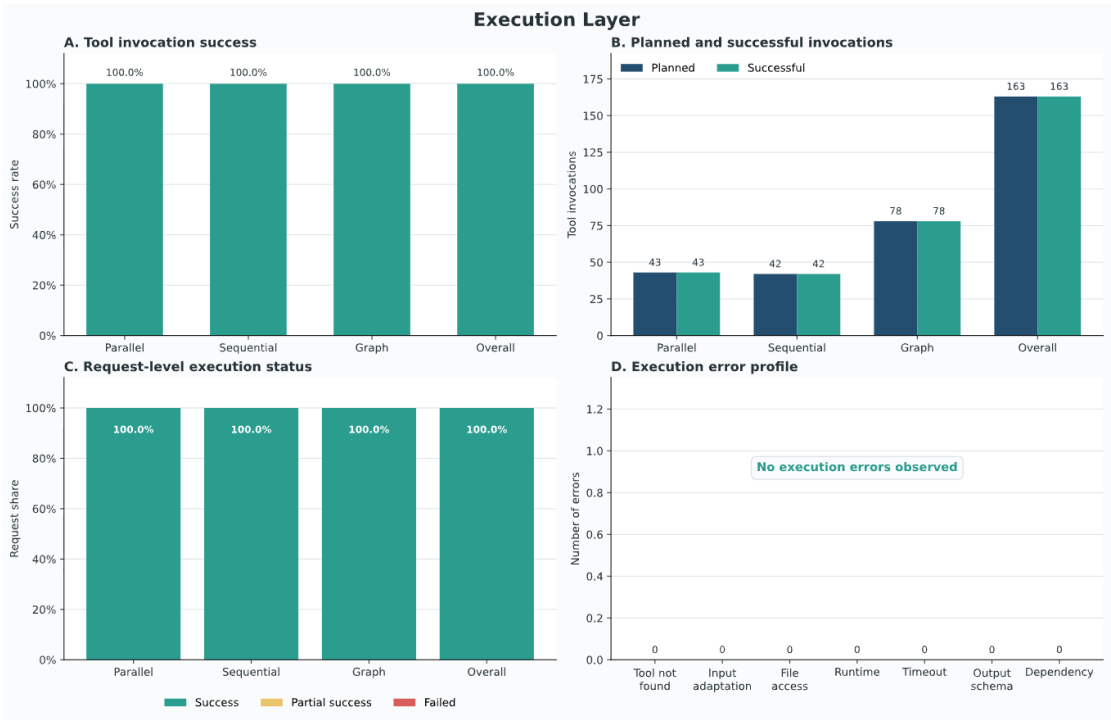


Figure 4 Performance of the execution layer.

The executor processed 163 planned tool calls in 60 complex execution cases. All 163 tool calls completed successfully, achieving a 100.0% success rate. At the request level, all 60 cases completed successfully without partial execution or failure.

4.5 End-to-End Performance and Baseline Comparison

This section evaluates end -to-end performance of the proposed system and compares

it with a pure monolithic baseline system and an iterative monolithic baseline system.

4.5.1 Overall End-to-End Comparison

The system was evaluated based on 100 test requests covering single-task, parallel, sequential, and graph-structured execution modes. The system fully completed 80 requests, partially completed 14 requests, and failed 6 requests. A 0-2 scoring scale was applied, 2 for full completion, 1 for partial completion, and 0 for failure. The system achieved an average completion rate of 87.0%.

In addition to the overall completion rate, the system also achieved 94.0% in basic pass rate, 81.0% in task resolution rate, and 88.9% in mandatory requirement coverage. These results indicate that the system is generally able to generate responses supported by usable execution evidence, although some responses may lack some necessary information or fail to fully resolve all requested tasks.

Compared to a pure monolithic baseline system, this system achieves a higher full completion rate: 80 requests were fully completed, compared to 76 for the pure monolithic system. The proposed system also achieves a higher task resolution rate of 81.0%, compared to 76.0% for the previous system. This indicates that a modular architecture helps the system complete more user requests at the task level, especially when requests require explicit decomposition and tool coordination.

However, the pure monolithic baseline system achieved a slightly higher average completeness score (88.0%), higher than the proposed system's 87.0%, and its

grounding pass rate (97.0%) was also higher than the proposed system's 94.0%. This difference indicates that the number of requests fully completed alone does not reflect all aspects of response quality. The proposed system scored 0 for some failure cases, while the purely monolithic system produced more partially acceptable answers. Therefore, the proposed system is more decisive in fully completing requests, but its failure cases have a greater impact on the average completeness score.

An initial comparison with the iterative monolithic baseline system shows that the iterative baseline system achieved the highest current completeness score, completing 94 requests with an average completeness of 97.0%. This shows that allowing a single agent to perform multiple inference and tool use steps can be very effective. However, this baseline model also uses tools more frequently, reflected in its higher rate of unnecessary tool use.

4.5.2 Performance by Single/Complex Request

The results show that request structure has a clear impact on the proposed system's end-to-end performance. It performed best on single-task requests, where it achieved a mean completeness of 93.10% and a groundedness pass rate of 100.0%. This is expected because single-task requests require less decomposition and fewer dependency decisions.

For parallel and sequential requests, the proposed system achieved the same mean completeness score of 87.5%. In parallel requests, the system fully completed 16 out of

20 cases, while in sequential requests it fully completed 15 out of 20 cases. Sequential requests also achieved a tool order accuracy of 100.0%, showing that the system was able to preserve required execution order when the dependency structure was relatively simple and linear.

Graph-structured requests were the most difficult category. The proposed system achieved a mean completeness of 80.65% on graph requests, with 23 fully completed, 4 partially completed, and 4 failed cases. This matches the planning-layer results in Section 4.4, which showed that dependency accuracy was lower for graph-structured plans compared to parallel and sequential plans. Graph requests usually contain multiple dependent branches and common intermediate results. A mistake in planning or evidence propagation may impact several downstream tasks simultaneously.

A different pattern was observed in the pure monolithic baseline. Its tool order accuracy was much lower on graph requests, but it was strong on completeness for single and sequential requests. For graph-structured requests, its required tool recall was 77.05% and tool order accuracy was just 33.01%. Thus, the pure monolithic system was less good at explicitly following complex tool dependencies, it often gave partially acceptable final answers.

The main advantage of the proposed system is therefore not that it is always producing the highest judged answer score, but that it is making the execution structure explicit. It still has weaker graph performance than it does with simpler request structures, but its modular design makes it easier to detect, analyze, and improve dependency errors.

4.5.3 Efficiency and Cost

Efficiency metrics include average latency, token usage, and number of LLM API calls per request. The proposed system takes an average time of 20.81 seconds per request, uses 10,140.96 tokens, and makes 2.83 API calls. The pure monolithic baseline system is more efficient and takes an average of 15.55 seconds per request with 6,458.52 tokens and 2.19 API calls. The iterative monolithic baseline system takes an average of 23.12 seconds per request, consuming 9,962.06 tokens and 2.61 API calls.

These results show that the proposed system has a significant overhead due to the additional inputs and outputs at each level of the modular pipeline. In particular, each of the understanding, planning and response generation modules requires separate calls to the LLM. This adds latency and token cost compared to a purely monolithic baseline system. The This

These results show that the proposed system has significant overhead because of the extra inputs and outputs needed at each level of the modular pipeline. In particular, each of the understanding, planning and response generation modules requires separate calls to the LLM. This incurs higher latency and token consumption compared to a purely monolithic baseline system. The overhead is especially large with complex requests, where the planning and response generation modules require more tokens, and the system should explicitly encode the task dependencies and the evidence of execution.

However, the additional cost brings advantages in interpretability and controllability.

The proposed system records intermediate states and these intermediate outputs enable

module-level evaluation and debugging of the system, which is difficult to achieve with a single monolithic hint.

Overall, the proposed system has higher computational overhead than a purely monolithic baseline system, while its latency and token consumption are comparable to iterative monolithic baseline systems.

4.6 Error Analysis

In this section, we present the main error patterns observed in the end-to-end evaluation. The proposed system successfully completed 80 out of 100 evaluation requests, partially completed 14 requests and failed on 6 requests. Errors were not evenly distributed over request structures. The observed errors can be categorized into four main types. These are errors of intent and routing, errors of planning and dependency, errors of tool execution or input adaptation, and errors of response generation.

4.6.1 Intent and Routing Errors

Some of the failures were due to wrong prediction of intent. The understanding module achieved a high overall intent F1, but some cases were ambiguous between closely related intents. For example, model-related requests sometimes involved both `printability_check` and `print_submission_check`, while the system predicted only one of them or added an extra related intent. While such errors were relatively rare. The corrected control accuracy was 97.0%, which shows that intent and routing errors were not the dominant source of end-to-end failure.

4.6.2 Planning and Dependency Errors

The most significant source of error is planning, especially for graph-structured requests. In standalone planning evaluations, graph planning performs worse on dependency accuracy than parallel and sequential planning. This pattern also appears in end-to-end results, where graph requests have the lowest average completeness score.

The challenge of graph requests lies in the fact that they typically require determining the dependencies between each detected intent. Unlike independent parallel requests, some tasks in a graph structure can only proceed after other tasks have completed and produced specific results. These dependencies are often not directly stated by the user in their natural language request. Therefore, the system not only needs to identify which intents the user includes, but also needs to further infer which tasks can be executed independently, which tasks must be executed sequentially, and whether the output of a task constitutes the input required for subsequent tasks. This means that graph request planning cannot rely solely on the identification results of a single intent, but requires a simultaneous understanding of the logical and information flow relationships between multiple subtasks. If the dependency determination is incomplete, missing dependency edges may cause downstream tasks to be executed before necessary intermediate results are generated, resulting in the tool lacking required input or producing unusable results. If necessary tasks are omitted during planning, subsequent execution chains will lack the information needed to achieve the user's goal, preventing the response generator from obtaining sufficient tool evidence to cover all parts of the request. Therefore, for

graph requests, accurately recovering task nodes and their dependencies is crucial to transforming complex user requests into a complete and executable workflow.

Some failures are related to the planning and validation phase. When the planning results fail to cover all identified intents, the validation module rejects the plan and returns a constraint that "each intent must be covered by at least one task." This indicates that the system can detect insufficient coverage in task planning and avoid executing potentially incomplete workflows. However, because the current recovery mechanism only returns validation error information without further triggering replanning or natural language interpretation, internal validation failures directly impact the quality of end-user responses.

4.6.3 Tool Execution and Input Adaptation Errors

The module-level performance evaluations on tool execution show that the executor is reliable when provided with a correct task plan. But in an end-to-end environment, execution errors still occur when tool parameters generated by earlier modules are invalid or insufficiently specified.

A typical problem arises during the 3D printed model inspection process. While the system can correctly identify tasks such as model size inspection, print time estimation, and rule checking, tool execution requires mapping the resource description in the user request to specific tool parameters. For example, a user might only mention "Bambu printer," while the model analysis tool requires a specific supported printer profile, such

as Bambu Lab X1C. If this mapping is not completed during the parameter adaptation phase, the tool will not be able to obtain the corresponding print size and print configuration, causing size inspection or print time estimation to fail. Since subsequent rule checks rely on these intermediate results, a local parameter parsing problem can further disrupt the entire task chain.

These mistakes go beyond simple execution-level mistakes. The planning or parameter adaption step is where the erroneous parameters come from, yet the executor reports the failures accurately. This implies that tool parameter validation has to be improved, particularly for instruments with restricted supported values or stringent input patterns.

4.6.4 Response Generation Errors

In certain cases that were only partially finished, the tool worked properly, but the outcome was not good enough. Incomplete evidence or a discrepancy between the user's request and the generated response were usually the causes of these issues. For instance, in certain tour guide creation scenarios, the system obtained pertinent facility or equipment data, but the resulting tour script was overly generic and did not sufficiently represent the user's particular requirements. In other instances, the response accurately represented the workshop findings that were retrieved, but it did not explicitly address whether the results were useful or satisfied the user's particular needs.

Another problem at the response level was the direct exposure of internal system messages. In some failed cases, the final response contained validation information or

process messages instead of a natural interpretation for the user. This shows that the response generation layer should include a more robust fallback mechanism to transform internal errors into user-friendly explanations.

Table 4 Summary of error patterns.

Error Type	Main Cause	Typical Effect
Intent/routing error	Missing or extra predicted intent	Wrong pipeline path or missing tool selection
Planning error	Missing task or dependency edge	Incomplete evidence for downstream response
Argument adaptation error	Invalid tool argument such as unsupported printer name	Tool failure and blocked dependent tasks
Response generation error	Weak evidence synthesis or exposed internal message	Partial answer or poor user-facing response

4.7 Discussion of Results

Evaluation results demonstrate that the proposed digital tutor can transform users' project needs into executable makerspace task flows, providing a structured orchestration mechanism for complex tool usage requests. The core value of this method lies in integrating request understanding, dependency-aware planning, tool invocation, and result-based response generation into a unified process. This enables the system not only to identify user needs but also to further determine the execution relationships between tasks and invoke the appropriate tools to complete the operation. Especially for complex requests involving multiple resources, tools, or consecutive steps, this architecture can organize the execution process based on task dependencies and integrate scattered tool results into a coherent final response, thereby establishing

a more direct and executable connection between student project goals and actual makerspace resources.

The architecture's use of clear intermediate stages to illustrate the reasoning and execution process is one of its main advantages. Instead of being concealed within a single end-to-end reasoning process, predicted intents, routing choices, scheduled tasks, dependency relationships, tool outputs, and the evidence utilized for response creation are kept as organized system states. This makes the behavior of the Digital Tutor inspectable at multiple levels and allows the contribution of individual modules to be evaluated independently. The module-level experiments confirm these individual stages can each be assessed as distinct capabilities, providing a clearer account of how an input is transformed into a final response.

The planning results further highlight the importance of modelling not only which operations are required, but also how those operations depend on one another. The proposed Planning Layer achieves strong task-selection and structural performance while supporting parallel, sequential, and general graph-structured workflows. This capability is particularly relevant to complex makerspace requests. Explicit dependency representation therefore extends the system beyond conventional intent-to-tool routing by allowing complex requests to be represented as executable task graphs.

The end-to-end comparison further shows that structured orchestration is particularly meaningful when requests require multiple coordinated operations. Explicit task decomposition and dependency-aware execution provide a systematic mechanism for

ensuring that the required operations are identified, ordered and connected to the final response. The comparison with monolithic tool-using baselines also reveals different orchestration profiles; more flexible iterative reasoning can achieve strong overall task completion, while the proposed architecture provides greater visibility into how tools are selected and coordinated and maintains explicit control over intermediate execution states. The value of the proposed approach therefore lies in making complex tool use an explicit and manageable process rather than leaving coordination entirely within an opaque reasoning loop.

This design is further characterized by efficiency measurements. The architecture distributes computing based on the processing pathway chosen for each request since comprehension, planning, execution, and response production are divided into discrete steps. While complicated requests receive the extra coordination needed to create and carry out multi-step workflows, simple requests can avoid dependency-aware planning and go straight to execution. Without forcing the same processing approach on each user request, this flexible framework offers a helpful foundation for implementing greater orchestration where it is most beneficial.

Above all, experimental results validate the effectiveness of the proposed method, demonstrating that a tool-call-oriented digital tutor orchestration mechanism can be achieved by explicitly modeling the dependencies between tasks. This architecture integrates user request understanding, task decomposition, tool execution, and results into a unified process, thereby more clearly establishing the connection between student

project needs and dispersed makerspace resources. Compared to simple single-step requests, this structured orchestration approach is more advantageous in handling complex tasks requiring the collaboration of multiple specialized tools or operations, coordinating the execution process according to the relationships between tasks, and ultimately forming a complete and coherent response.

Chapter 5 Conclusion

5.1 Summary of the Study

This work designed, develop and evaluated a digital tutor system for the Innovation Wing at the University of Hong Kong. The system seeks to assist students in making connections between project-related requirements and pertinent facilities, workshops, equipment, and technical assistance. The suggested design divides task planning, tool execution, response creation, and need comprehension and control. While complicated needs are broken down into several actions with clear relationships, simple needs can be executed directly.

To assess the system both end-to-end and at the module level, we employed a special dataset with 100 Innovation Center requirements. In order to assess the system's efficacy and computational cost, we also contrasted it with a baseline system that used a single tool.

5.2 Main Findings

The understanding and control pipeline performs well in intent recognition and routing. For graph-structured requests, dependency construction is still difficult, but the planning layer is quite effective at choosing tasks. Planning, argument preparation, or evidence processing may be the root cause of many end-to-end failures, as the execution layer is quite dependable in producing the right task graph.

Modular orchestration is not always better than monolithic agents, according to baseline comparisons. Although the suggested solution necessitates additional model invocation, tokens, and latency, it offers greater explicit control over job breakdown, dependencies, and tool utilization. These findings demonstrate that requests requiring numerous tools, interim outputs, and explicit dependency management are best served by modular orchestration.

5.3 Limitations

The study has several limitations. First, the evaluation dataset may not fully reflect the diversity of actual student inquiries because it only includes 100 questions from a single makerspace context. An LLM helped create the original examples, which were then examined and updated by a current Innovation Wing tutor to increase their applicability and realism. While this method offers domain-specific test cases, it cannot ensure that every scenario that might occur in real life use is covered. There is relatively little linguistic coverage in the dataset.

Second, the evaluation that is currently in place is mostly functional and benchmark-based. Although the Digital Tutor has not yet been implemented and assessed as part of the regular Innovation Wing scenarios, it evaluates the system's ability to comprehend requests, create plans, carry out tools, and produce grounded responses under controlled experimental conditions. As a result, the current study does not directly assess how actual students and employees would engage with the system over time or how it would

impact resource discovery, project advancement, and support procedures in real-world settings.

The system may also be affected by upstream intent-recognition and planning errors, with limited recovery mechanisms when generated plans or tool arguments require correction. Furthermore, compared to simpler execution pathways, the modular architecture necessitates numerous processing stages, which adds to the computational cost. These restrictions point to specific avenues for further assessment, including more extensive real-world case collection, multilingual testing, staff and student deployment, and more robust recovery mechanisms for multi-step workflows.

5.4 Future Work

Future research should improve the suggested system's resilience and flexibility even more. Strengthening graph dependency planning, enhancing tool validation, and implementing automated recovery mechanisms for unsuccessful planning or tool execution are some possible paths. By applying lightweight processing to straightforward requests and only turning on more formal planning when the complexity of the request demands it, orchestration techniques can also become more adaptable. A more thorough assessment of system performance will also be made possible by larger-scale, more varied benchmarking, repeated experimental runs, and a greater variety of evaluation scenarios.

Beyond technical evaluation, future work should explore the practical impact of digital

mentors through user research with students and faculty at the innovation center. The improved system can be deployed in real-world makerspace environments and compared with existing support methods, such as website-based information access, equipment instructions, and mentor or technician assistance. Student feedback can be collected to assess the system's perceived usefulness, ease of use, accessibility of support, confidence in identifying suitable resources, and overall learning experience. Task-based measurements, such as the time required to find relevant facilities or determine the next steps in a project, can further reveal whether digital mentors can improve students' ability to utilize makerspace resources. Such research helps determine whether the technical improvements achieved by the proposed architecture can translate into meaningful educational benefits in practice.

Appendix A. Project Repository

The source code and implementation associated with this thesis are available at:

<https://github.com/yourname/your-repository>

References

- [1] V. Sukackè *et al.*, "Towards active evidence-based learning in engineering education: A systematic literature review of PBL, PjBL, and CBL," *Sustainability*, vol. 14, no. 21, p. 13955, 2022.
- [2] C. S. E. Jamison, J. Fuher, A. Wang, and A. Huang-Saad, "Experiential learning implementation in undergraduate engineering education: a systematic search and review," *European Journal of Engineering Education*, vol. 47, no. 6, pp. 1356–1379, 2022.
- [3] F. B. Mota, B. P. Cabral, L. A. M. Braga, and R. M. Lopes, "Mapping the global research on project-based learning: a bibliometric and network analysis (2014–2024)," in *Frontiers in Education*, 2025, vol. 10: Frontiers Media SA, p. 1522694.
- [4] V. Wilczynski, "Academic maker spaces and engineering design," in *2015 ASEE annual conference & exposition*, 2015, pp. 26.138. 1–26.138. 19.
- [5] J. Bouwma-Gearhart, Y. H. Choi, C. A. Lenhart, I. Villanueva, L. S. Nadelson, and E. Soto, "Undergraduate students becoming engineers: The affordances of university-based makerspaces," *Sustainability*, vol. 13, no. 4, p. 1670, 2021.
- [6] T. U. o. H. Kong. "HKU Engineering's brand new maker space for students: Tam Wing Fan Innovation Wing—One of the largest maker spaces in Asia." https://www.hku.hk/press/news_detail_21321.html (accessed August 4, 2026).
- [7] I. Villanueva Alarcón, R. J. Downey, L. Nadelson, Y. H. Choi, J. Bouwma-Gearhart, and C. Tanoue, "Understanding equity of access in engineering education making spaces," *Social sciences*, vol. 10, no. 10, p. 384, 2021.
- [8] M. E. Andrews and A. Boklage, "Alleviating barriers facing students on the boundaries of STEM makerspaces," *Education Sciences*, vol. 14, no. 7, p. 772, 2024.
- [9] M. W. Alemán, M. E. Tomko, J. S. Linsey, and R. L. Nagel, "How do you play that makerspace game? An ethnographic exploration of the habitus of engineering makerspaces," *Research in engineering design*, vol. 33, no. 4, pp. 351–366, 2022.
- [10] S. Hobert and F. Berens, "Developing a digital tutor as an intermediary between students, teaching assistants, and lecturers," *Educational technology research and development*, vol. 72, no. 2, pp. 797–818, 2024.
- [11] Z. Chu *et al.*, "LLM Agents for Education: Advances and Applications," *EMNLP (Findings)*, pp. 13782–13810, 2025.
- [12] T. Feng, S. Liu, and D. Ghosal, "Courseassist: Pedagogically appropriate ai tutor for computer science education," in *Proceedings of the 2024 on ACM Virtual Global Computing Education Conference V. 2*, 2024, pp. 310–311.

- [13] T. Wang *et al.*, "Llm-powered multi-agent framework for goal-oriented learning in intelligent tutoring system," in *Companion Proceedings of the ACM on web conference 2025*, 2025, pp. 510–519.
- [14] V. Hudeček and O. Dušek, "Are large language models all you need for task-oriented dialogue?," in *Proceedings of the 24th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, 2023, pp. 216–228.
- [15] V. Dongre, X. Yang, E. C. Acikgoz, S. Dey, G. Tur, and D. Hakkani-Tur, "Respect: Harmonizing reasoning, speaking, and acting towards building large language model-based conversational ai agents," in *Proceedings of the 15th international workshop on spoken dialogue systems technology*, 2025, pp. 72–102.
- [16] A. Gupta, A. Ravichandran, Z. Zhang, S. Shah, A. Beniwal, and N. Sadagopan, "Dard: A multi-agent approach for task-oriented dialog systems," *arXiv preprint arXiv:2411.00427*, 2024.
- [17] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face," *Advances in Neural Information Processing Systems*, vol. 36, pp. 38154–38180, 2023.
- [18] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, "Rewoo: Decoupling reasoning from observations for efficient augmented language models," *arXiv preprint arXiv:2305.18323*, 2023.
- [19] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, 2024, vol. 2024, pp. 23247–23275.
- [20] Y. Wu, T. Yue, S. Zhang, C. Wang, and Q. Wu, "Stateflow: Enhancing llm task-solving through state-driven workflows," *arXiv preprint arXiv:2403.11322*, 2024.
- [21] Y. Zhang, X. Liu, and C. Xiao, "Metaagent: Automatically constructing multi-agent systems based on finite state machines," *arXiv preprint arXiv:2507.22606*, 2025.
- [22] G. Zhang, L. Niu, J. Fang, K. Wang, L. Bai, and X. Wang, "Multi-agent architecture search via agentic supernet," *arXiv preprint arXiv:2502.04180*, 2025.
- [23] J. Su *et al.*, "Difficulty-aware agent orchestration in LLM-powered workflows," *arXiv e-prints*, p. arXiv: 2509.11079, 2025.